

Mini Projet B – Comparaison de méthodes d'intégration numérique

L'objectif de ce projet est d'implémenter, comparer et analyser différentes méthodes d'intégration numérique appliquées à une fonction polynomiale. Trois approches ont été étudiées :

- la méthode des rectangles ;
- la méthode des trapèzes ;
- la méthode de Simpson.

Pour chacune de ces méthodes, différentes implémentations ont été réalisées à l'aide de Python de base, NumPy et SciPy lorsque cela était applicable. Les performances ont ensuite été comparées en termes de précision et de temps d'exécution.

1. Fonction étudiée

La fonction utilisée pour les expérimentations est $f(x) = 1 + 2x + 3x^2 + 4x^3$ sur l'intervalle $[0,10]$. La primitive exacte de cette fonction est :

$$F(x) = x + x^2 + x^3 + x^4$$

La valeur analytique de référence est donc :

$$F(10) - F(0) = 11110$$

Cette valeur a été utilisée pour calculer les erreurs des différentes méthodes numériques.

2. Méthodes d'intégration étudiées

2.1. Méthode des rectangles

La méthode des rectangles approxime l'aire sous la courbe par une succession de rectangles. La hauteur de chaque rectangle est déterminée à partir de la valeur de la fonction au milieu de chaque sous-intervalle.

Cette méthode est simple à implémenter mais demeure généralement moins précise que les méthodes des trapèzes et de Simpson.

2.2. Méthode des trapèzes

La méthode des trapèzes remplace chaque rectangle par un trapèze dont les côtés suivent les valeurs de la fonction aux extrémités de chaque segment.

Cette approche améliore généralement la précision des résultats puisque l'évolution de la fonction est mieux représentée.

2.3. Méthode de Simpson

La méthode de Simpson utilise une interpolation quadratique entre les points de calcul afin d'obtenir une approximation plus fidèle de la courbe.

Pour les polynômes de degré inférieur ou égal à trois, cette méthode permet théoriquement de retrouver la valeur exacte de l'intégrale.

3. Implémentation

Les algorithmes ont été développés en Python et organisés dans des modules distincts.

Trois types d'implémentation ont été étudiés :

- Python de base utilisant des boucles classiques ;
- NumPy utilisant des opérations vectorisées ;
- SciPy utilisant des fonctions préprogrammées d'intégration.

Un programme principal permet :

- de calculer les intégrales ;
- d'évaluer les erreurs ;
- de mesurer les temps d'exécution ;
- de générer les graphiques de convergence ;
- de comparer les performances des différentes méthodes.

Des tests unitaires ont également été développés avec Pytest afin de vérifier automatiquement la validité des résultats obtenus.

4. Résultats

Pour un découpage de ($n = 100$) segments, les résultats obtenus sont présentés ci-dessous :

Méthode	Résultat	Erreur
Rectangle Python	11109.475	0.525
Rectangle NumPy	11109.475	0.525
Trapèze Python	11111.050	1.050
Trapèze NumPy	11111.050	1.050
Trapèze SciPy	11111.050	1.050
Simpson Python	11110.000	3.64×10^{-12}
Simpson NumPy	11110.000	1.82×10^{-12}
Simpson SciPy	11110.000	0

Les résultats démontrent que toutes les méthodes convergent vers la valeur exacte, mais avec des niveaux de précision différents.

5. Analyse des graphiques

5.1. Convergence des méthodes

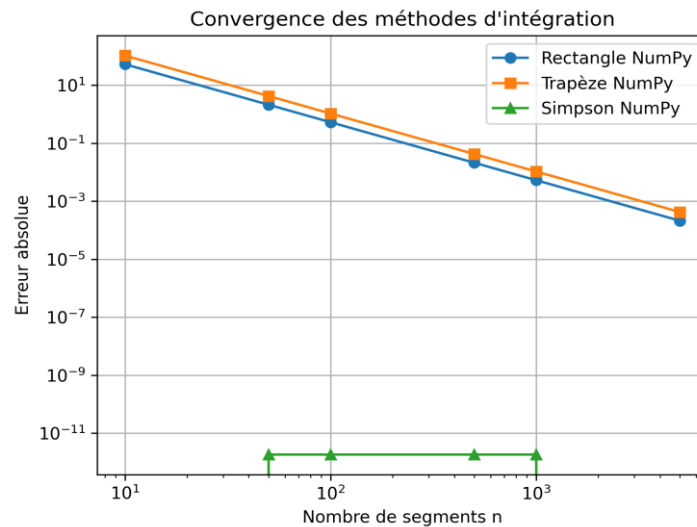


Figure 1: Convergence des méthodes d'intégration

Évolution de l'erreur absolue en fonction du nombre de segments utilisés pour les méthodes des rectangles, des trapèzes et de Simpson.

L'analyse du graphique et des résultats numériques montre que l'erreur diminue lorsque le nombre de segments augmente. Contrairement à une idée reçue, pour cette fonction spécifique, la méthode des rectangles (qui utilise le point milieu) offre une précision deux fois supérieure (erreur de 0.525) à celle de la méthode des trapèzes (erreur de 1.050). Ce phénomène mathématique s'explique par le fait que l'évaluation au point milieu compense souvent mieux les variations de la courbe que la simple liaison des extrémités utilisée par les trapèzes. Enfin, la méthode de Simpson se démarque comme étant de loin la plus performante, puisqu'elle atteint pratiquement la précision machine (erreur nulle) dès les premières valeurs de n , grâce à son interpolation quadratique parfaitement adaptée à ce polynôme de degré 3.

5.2. Temps d'exécution

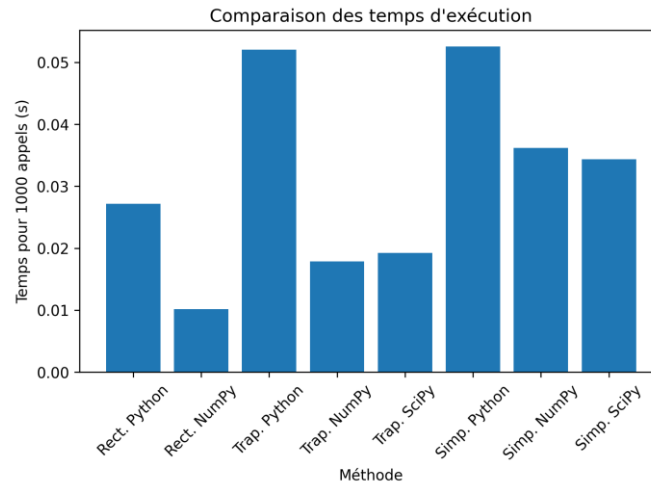


Figure 2 : Comparaison des temps d'exécution

Temps requis pour exécuter 1000 appels de chacune des méthodes d'intégration étudiées.

Les mesures montrent que les implémentations utilisant NumPy sont généralement plus rapides que les versions Python classiques.

Les fonctions préprogrammées de SciPy offrent également d'excellentes performances tout en simplifiant le développement.

Ces résultats confirment l'intérêt des bibliothèques scientifiques pour le calcul numérique.

6. Validation

Les implémentations ont été validées à l'aide de tests unitaires réalisés avec Pytest.

Les tests vérifient que les différentes méthodes retournent les valeurs attendues pour le polynôme étudié.

L'exécution de la suite de tests a donné le résultat suivant :

- 8 tests exécutés ;
- 8 tests réussis ;
- aucune erreur détectée.

Cette validation confirme le bon fonctionnement des implémentations développées.